20 Agentic Design Patterns (Part 2)

A comprehensive guide to 20 agentic design patterns that separate pros from beginners, based on a Google engineer's 400-page book. Practical patterns you can use today with plain English explanations.

Back

Navigation

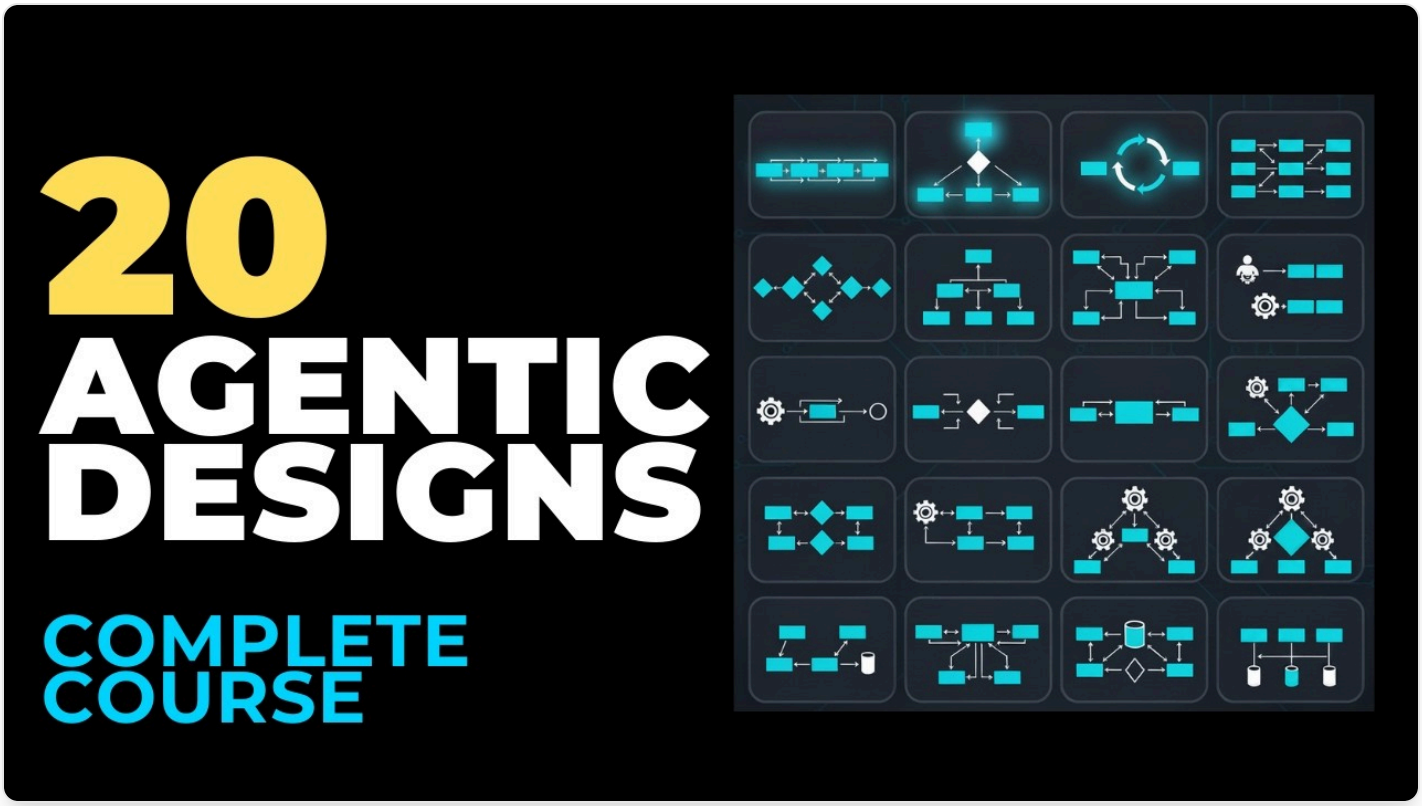
This guide is split into 4 parts for better performance:

- Part 1: Chapters 1-5 - Prompt Chaining, Routing, Parallelization, Reflection, Tool Use
- Part 2: Chapters 6-10 - Planning, Multi-Agent Collaboration, Memory Management, Learning and Adaptation, Goal Setting and Monitoring
- Part 3: Chapters 11-15 - Exception Handling and Recovery, Human in the Loop, Knowledge Retrieval (RAG), Inter-Agent Communication, Resource-Aware Optimization
- Part 4: Chapters 16-20 - Reasoning Techniques, Evaluation and Monitoring, Guardrails and Safety Patterns, Prioritization, Exploration and Discovery

Originally video talking about agentic systems.

You can help out the author who broke down the 400-page manual published by the Google engineer here.

Link not affiliated.



Chapter 6: Planning

TLDDR: Creating a step-by-step plan for big goals with checkpoints. This is what I do personally when I use things like Claude Code or Cursor. I don't write code for like 40 minutes or AI doesn't write code for 40 to 50 minutes. I plan and plan and plan until it's ready to go and I know exactly

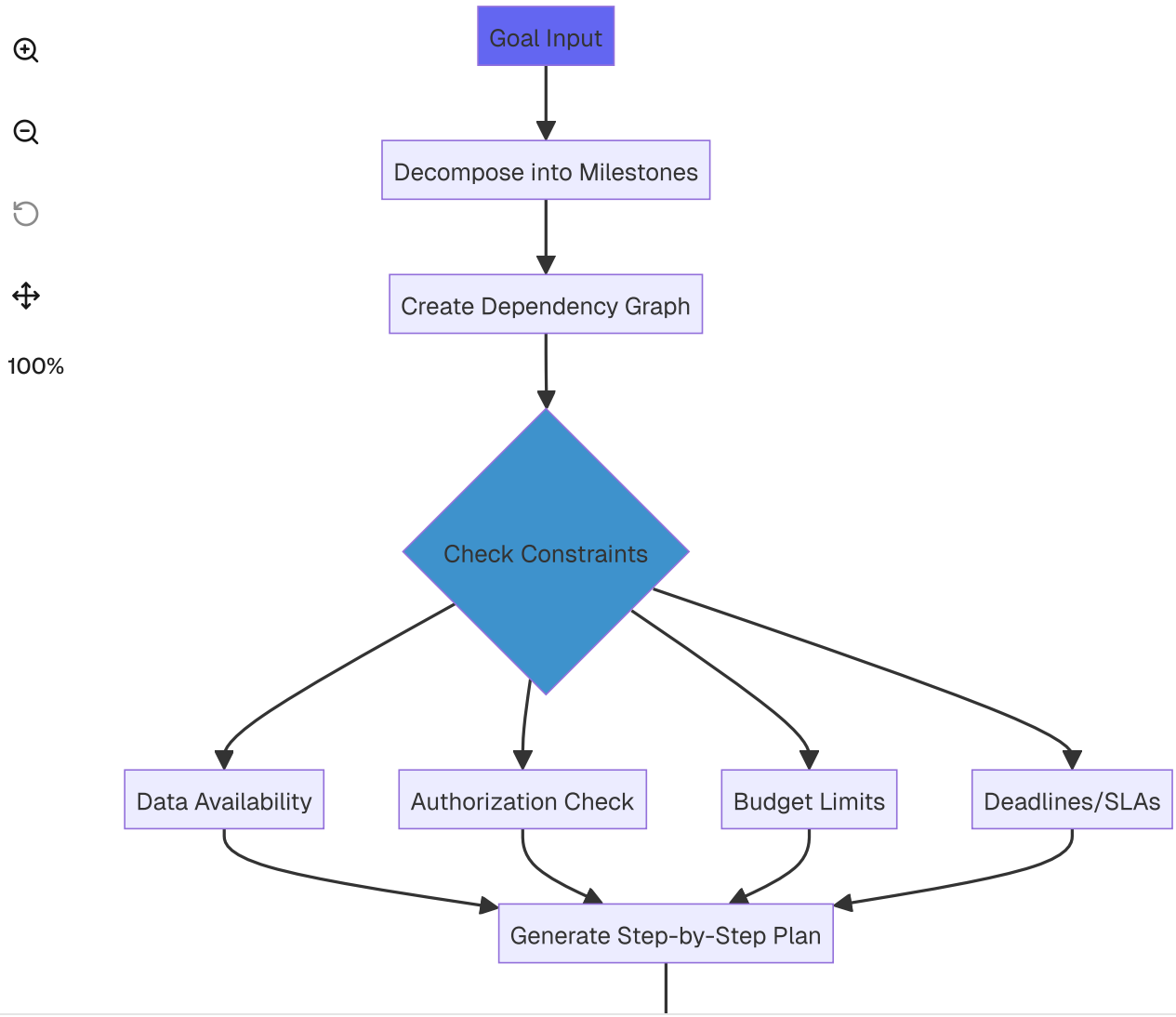
When to Use Where It Fits

When to Use

- **Complex multi-step projects**: When tasks have multiple dependencies and phases
- **Goal-oriented workflows**: When working toward specific, measurable objectives
- **Resource-constrained operations**: When managing budgets, time, or computational limits
- **Uncertain environments**: When adaptability to changing conditions is needed
- **Collaborative tasks**: When coordinating multiple agents or tools
- **Long-running processes**: When tasks span extended timeframes

How It Works

MermaidZoom MMD



How does planning actually work?

Break down your big goal into milestones, then create a dependency graph

Dependency graph?

Shows what needs to happen first. Like you can't deploy code before you write it

Check your constraints - do you have the data? Authorization? Budget? Time?

Ok constraints pass

Generate step-by-step plan, assign agents and tools to each step

Different from prompt chaining?

Yeah. Prompt chaining carries output between steps. Planning just executes sequentially - step 1, step 2, step 3... until step N

What if something fails?

Analyze the blocker. New info? Replan. Edge case? Human in the loop. Otherwise escalate and handle it

And if it works?

Track progress, check acceptance criteria, you're done

Pros Cons

Pros

- **Strategic execution:** Transforms reactive agents into proactive planners
- **Dependency management:** Handles complex task interdependencies

- **Progress visibility**: Clear tracking of milestone completion
- **Risk mitigation**: Early identification of blockers and issues
- **Reusability**: Plans can be templated and reused

Real-World Examples

Software Development | Marketing Campaign | Research Project | Data Migration | Product Launch | Cor

Software Feature Development

- Requirements analysis and design
- Development task breakdown
- Testing strategy planning
- Deployment scheduling
- Documentation preparation
- Rollback planning

Chapter 7: Multi-Agent Collaboration

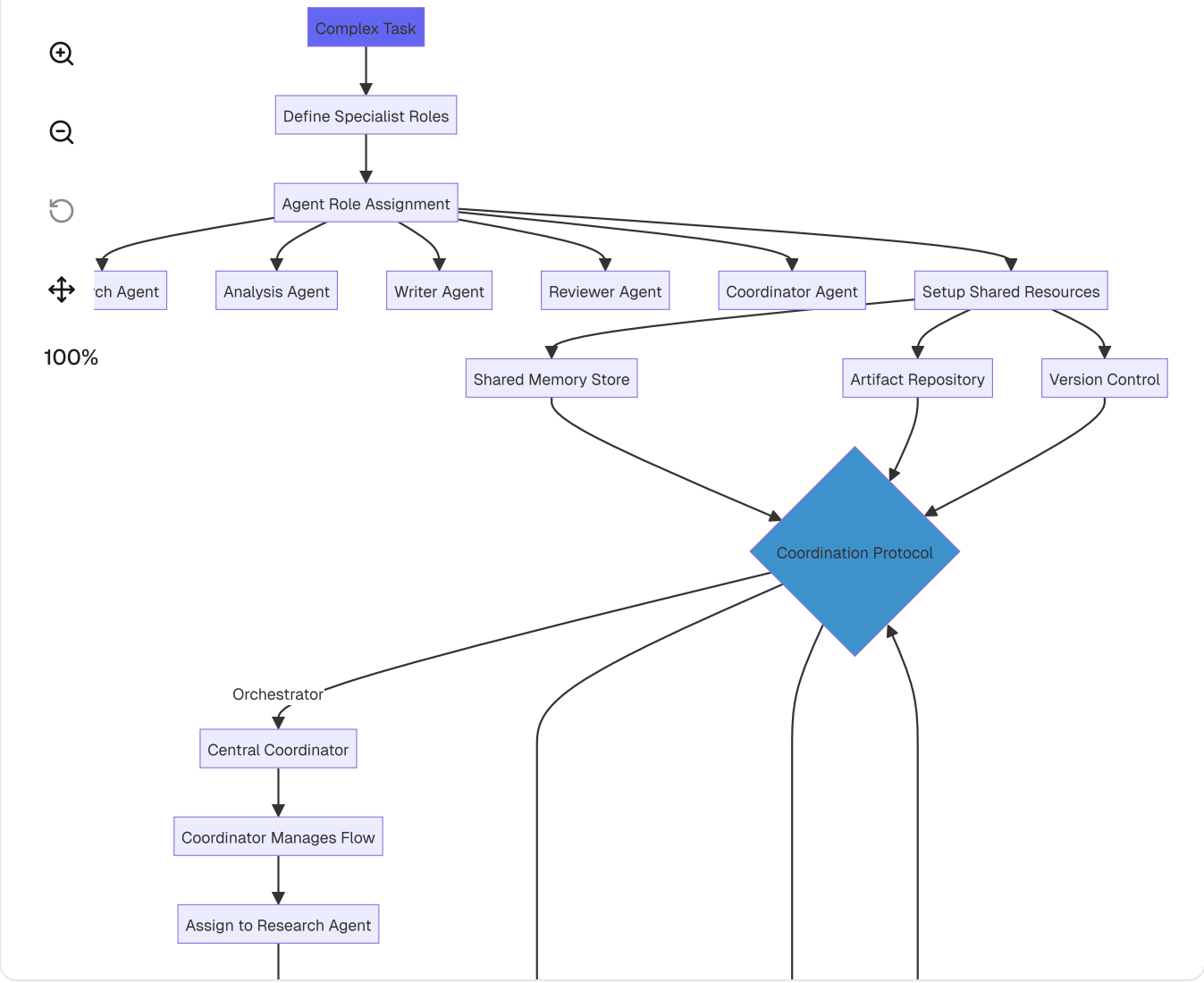
TLDDR: Multiple specialized agents working together on different parts of a complex task coordinated by some central manager. In many cases they share a common memory which is important here because if you share a common memory then your memory mechanism whatever that is an MCP server any form of function has to be well structured so that all the memories don't overlap but you focus on the proper memories that need to persist. Think of it like having a film crew where the director coordinates while camera, sound, and lighting specialists each handle their part sharing the same script and timeline.

When to Use

- **Complex, multi-faceted problems**: Tasks requiring diverse expertise
- **Parallel workstreams**: When subtasks can be handled simultaneously
- **Specialized knowledge requirements**: Different aspects need different skills
- **Scale and efficiency**: Large projects benefiting from division of labor
- **Quality through specialization**: When expertise depth matters
- **Iterative refinement**: Tasks requiring multiple perspectives

How It Works

MermaidZoom MMD



How does multi-agent collaboration work?

Define specialist roles - research agent, analysis agent, writer agent, etc. One agent decides which agent to use

How do they share info?

How do they coordinate?

Coordinator manages flow. Like a Jira board - tags tickets to agents, checks acceptance criteria before moving to next stage

What if it fails?

Run simulation to find where coordination failed, loop back, set max retries so it doesn't go forever

Pros Cons

Pros

- **Specialization benefits**: Each agent optimized for specific tasks
- **Parallel processing**: Multiple agents work simultaneously
- **Scalability**: Easy to add new specialist agents
- **Modularity**: Agents can be developed and updated independently
- **Robustness**: Failure of one agent doesn't crash entire system
- **Knowledge separation**: Clear boundaries between domains
- **Quality improvement**: Multiple perspectives and validation steps

Real-World Examples

News Production Investment Analysis E-commerce Launch Legal Review Bug Resolution Paper Re

- News Gatherer Agent: Collects breaking news from sources
- Fact Checker Agent: Verifies claims and sources
- Writer Agent: Drafts article with proper structure
- Editor Agent: Improves clarity and style
- SEO Agent: Optimizes for search engines
- Publisher Agent: Formats and publishes to CMS

Chapter 8: Memory Management

TLDDR: Classifying incoming information as short-term conversation, episodic events, or long-term knowledge. You store each type appropriately with metadata like recency and relevance. This is exactly how your brain keeps track of things briefly, some like specific memories or permanent knowledge, things that you will never forget. One thing I would say here is that there are so many tools and MCP servers trying to solve this. And I've yet to find something perfect because I noticed that depending on what you're trying to build agentically, memory management is really contextually specific on what you're trying to remember and what is not worth remembering.

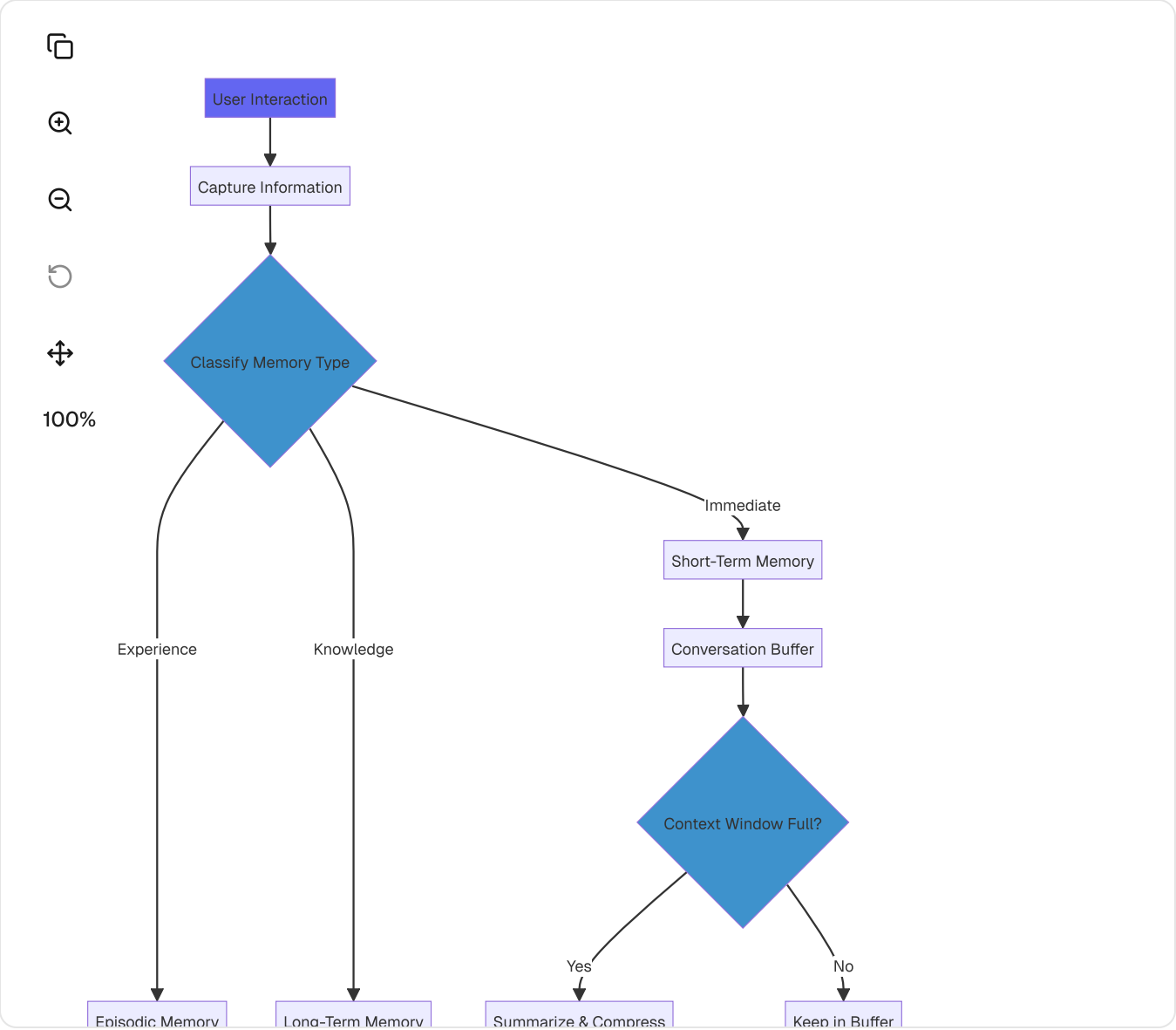
When to Use | Where It Fits

When to Use

- **Conversational continuity**: Maintaining context across interactions
- **Personalization**: Remembering user preferences and history
- **Learning systems**: Accumulating knowledge over time
- **Complex workflows**: Tracking state across multiple steps
- **User sessions**: Managing multi-turn conversations
- **Knowledge accumulation**: Building domain expertise over time

How It Works

MermaidZoom MMD



How does memory management work?

Classify info into three types: short-term (conversation), episodic (events), long-term (knowledge)

Short-term stays in buffer until context window is full, then compress. Episodic and long-term get indexed with metadata

What metadata?

Recency score, access frequency, topic tags. Helps you find relevant memories later

How do you retrieve memories?

Query memory store, filter by role, time range, or topic match. Check if expired, then load to context

What about privacy?

Check for sensitive data, redact if needed, then write to memory. Update and continue

Pros Cons

Pros

- **Context preservation**: Maintains conversation continuity
- **Personalization**: Enables tailored responses based on history
- **Learning capability**: Improves performance through experience
- **Efficiency**: Avoids repeating previous work
- **User experience**: More natural, human-like interactions

Real-World Examples

Customer Support | Shopping Assistant | Code Assistant | Medical Bot | Education Tutor | Project Manag

Customer Support System

- Short-term: Current conversation context
- Episodic: Previous support tickets and resolutions
- Long-term: Customer preferences and history
- Automatic summarization of long conversations
- Privacy-compliant data retention policies

Chapter 9: Learning and Adaptation

TLDDR: Collecting feedback from user corrections, ratings, and outcomes. You want to clean and validate the data to remove noise and then you use it to update prompts, policies, or examples. It's like adjusting a recipe based on customer feedback and taste tests. So essentially, you'd have some form of system operation and you collect feedback from a feedback source. That could be some form of correction from a user, quality ratings, automated evaluations, or some form of rubric or task outcomes.

When to Use | Where It Fits

When to Use

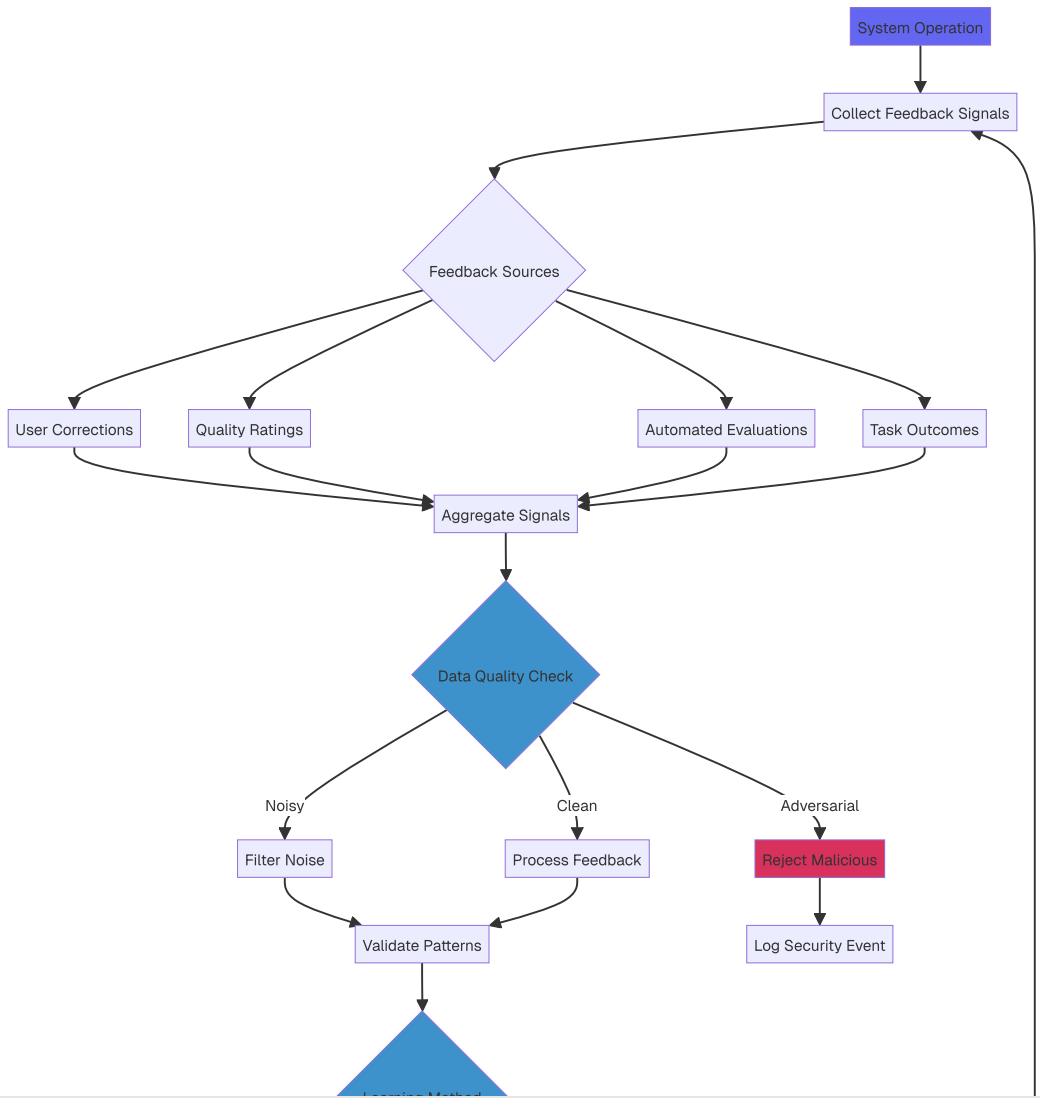
- **Performance optimization**: When system needs to improve over time
- **User personalization**: Adapting to individual user preferences

- **Dynamic environments**: Adapting to changing conditions
- **Feedback incorporation**: When user corrections are available

How It Works

MermaidZoom MMD

100%



How does learning and adaptation work?

Collect feedback from user corrections, ratings, automated evaluations, task outcomes

Then what?

Aggregate signals, check data quality. Filter noise, reject malicious feedback, process clean data

How do you learn from it?

Update prompts, adjust policies, add to few-shot examples, update preferences. Or fine-tune the model

What if you learn something wrong?

A/B test changes, monitor performance. If it improves, deploy. If it regresses, rollback. If neutral, keep learning

Example of learning wrong?

Like a restaurant review system that learns fake reviews saying there are cockroaches everywhere, then warns customers about non-existent cockroaches

Pros Cons

Pros

- **Continuous improvement**: System gets better with use

- **Efficiency gains**: Optimizes common patterns over time
- **Robustness**: Adapts to changing requirements
- **User satisfaction**: Better alignment with expectations
- **Knowledge retention**: Preserves learned improvements

Real-World Examples

Customer Support | Code Review | Content Writing | Financial Advisory | Medical Diagnosis | E-commerc

Customer Support Chatbot

- Learns from agent takeovers and corrections
- Adapts responses based on satisfaction ratings
- Updates FAQ answers from successful resolutions
- Improves intent classification from mislabeled queries
- Personalizes tone based on customer feedback

Chapter 10: Goal Setting and Monitoring

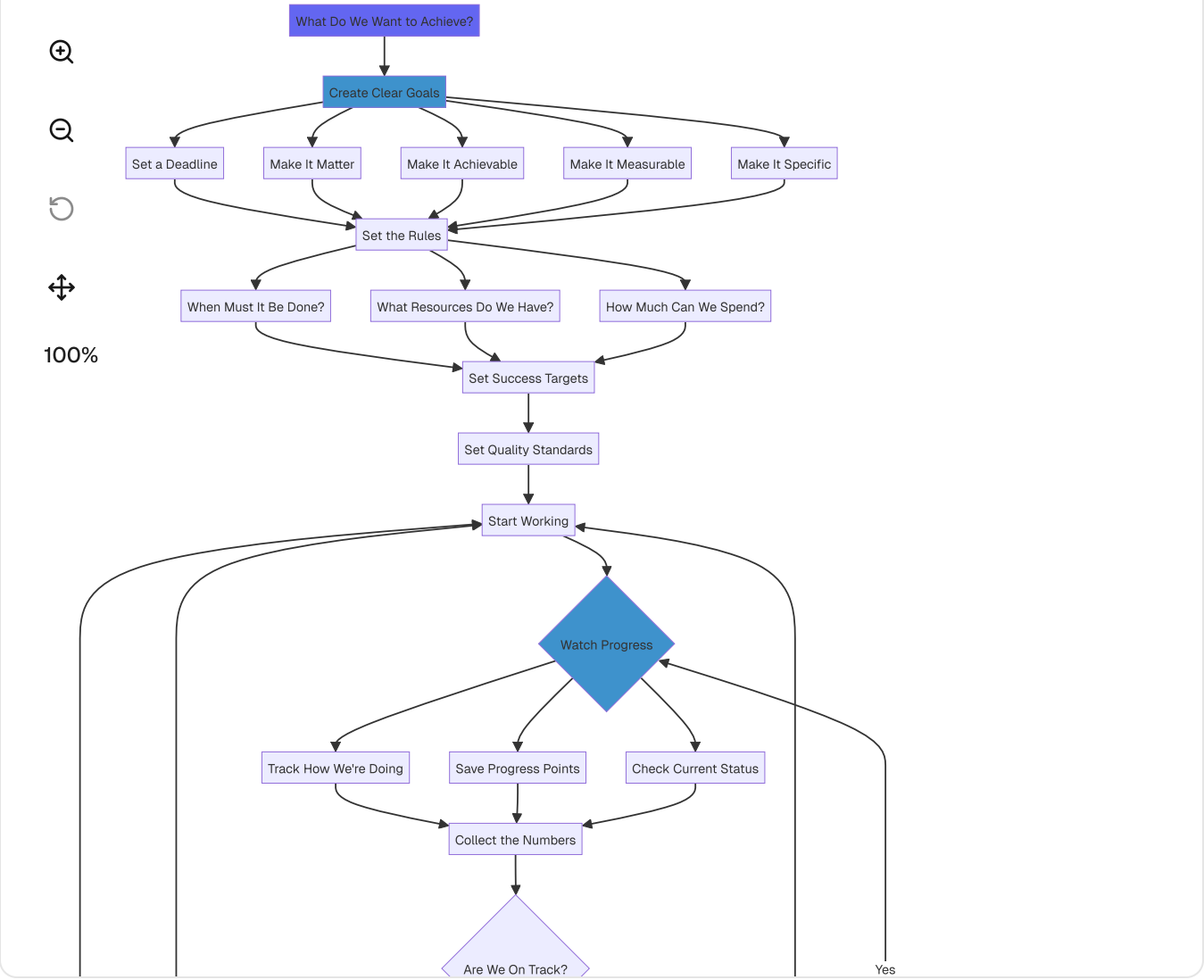
TLDDR: Defining specific measurable goals. A lot of times they call these SMART goals. Specific, measurable, achievable, realistic, and time-based. You have measurable goals with deadlines and budgets and then as work progresses, you continuously monitor metrics and compare to targets. It's like having a GPS that sets a destination, monitors your progress, and then recalculates when you're off course.

When to Use | Where It Fits

- **Autonomous operations**: When agents work independently toward objectives
- **Complex projects**: Multi-step tasks requiring progress tracking
- **Resource management**: When operating within constraints
- **Performance optimization**: Achieving specific measurable outcomes
- **Compliance requirements**: Meeting SLAs and quality standards
- **Strategic execution**: Aligning agent actions with business goals

How It Works

MermaidZoom MMD



How does goal setting and monitoring work?

Create SMART goals - specific, measurable, achievable, relevant, time-bound.
Set rules, constraints like budget, resources, deadlines

Then what?

What if you're off track?

Sound the alarm, find out why. Then adjust plan, get more resources, or change the goal

How do you know if you succeeded?

Check if goal achieved. If yes, create report. If no, check if you still have budget. If yes, keep monitoring. If no, stop or get more

Pros Cons

Pros

- **Purpose-driven**: Agents work toward clear objectives
- **Self-assessment**: Continuous evaluation of progress
- **Adaptability**: Dynamic adjustment to changing conditions
- **Accountability**: Clear metrics and success criteria
- **Resource efficiency**: Optimal allocation based on priorities
- **Early warning**: Proactive detection of issues
- **Measurable outcomes**: Quantifiable success metrics

Real-World Examples

Sales Automation Content Publishing DevOps Pipeline Customer Service Marketing Campaign Su

- Monthly revenue targets with daily tracking
- Lead conversion rate goals
- Customer acquisition cost limits
- Activity metrics (calls, emails, meetings)
- Automatic escalation for at-risk deals
- Performance dashboard generation